

Fabric Data Agent Workshop Labs

Participant guide | October 2026

Build a manufacturing operations data agent, improve its business understanding, evaluate its answers, and extend it with a Lakehouse source.

This guide follows the Microsoft Fabric Data Agent Workshop's three labs. Questions and actions are provided in execution order. Use the **Expected behavior** and **If different** notes to assess your own results: wording, generated queries, visual formats, and latency can vary between runs.

Starting point: SetupDataAgentJumpstart has already run successfully. Do not reinstall the Jumpstart or rerun setup just to begin this guide.

Formats: [Markdown](#) | [Download the PDF](#)

Contents

- Before you start
- Lab 1: Creating and optimizing data agents
- Lab 2: Programmatic evaluation of data agents
- Lab 3: Adding multiple data sources
- Workshop completion checklist
- Troubleshooting
- Sources and license

Before you start

Confirm the prepared environment

1. Sign in to [Microsoft Fabric](#) with the account provided for the workshop, then open your assigned workspace.
2. Open the `getting-started-data-agents` folder.
3. Confirm that both a semantic model and a report exist for `ManufacturingOps` and `ManufacturingOpsAIReady`.
4. Confirm that the following six notebooks are present.

Notebook	Role in the workshop
<code>SetupDataAgentJumpstart</code>	Already completed: imports and configures the populated models and reports.
<code>RefreshSemanticModel</code>	Optional maintenance; not required simply because the workshop is starting.
<code>JudgeCalibration</code>	Calibrates and registers the evaluation judge in Lab 2.
<code>EvaluateDataAgent</code>	Evaluates your AI-ready agent in Lab 2.
<code>BuildOpsRefData</code>	Builds the Lakehouse and its SQL-facing objects in Lab 3.
<code>CreateMultiSourceDataAgent</code>	Configures and publishes your multi-source agent in Lab 3.

Expected behavior: two semantic models, two reports, and six notebooks are available. The models contain cached data. Notebook presence alone does not prove that setup completed: open the two reports and confirm that their visuals load data rather than displaying connection errors.

Both reports open on a blank **Page 1**. Use the page tabs at the bottom:

- `ManufacturingOps` report: open the **info** page. It shows the data-as-of card.
- `ManufacturingOpsAIReady` report: open the **Verified Answer** page. It shows scrap rate by manufacturer.

If different: ask the facilitator to finish or repair setup. Do not create duplicate models, change capacity, or rerun setup over other participants' work. A successful refresh is not a prerequisite when the populated imported models already work.

Facilitator note — known setup failure: in the tested notebook version, the first `SetupDataAgentJumpstart` run can stop at the `DataFolder` parameter update with **HTTP 403**. The cause is one request whose `Authorization` header contains a literal placeholder instead of the token. In that cell, set the header to `"Bearer " + power_bi_token` (the notebook's existing Power BI token variable), or ask notebook Copilot to fix the error, then rerun. The second run completes both parameter updates and the final data checks. No permission change is needed.

Access and optional features

The facilitator normally prepares the paid Fabric capacity, Power BI Pro license, workspace Contributor access, and required Data Agent/Copilot tenant settings. If **New item > Data agent** is unavailable, stop and ask the facilitator to check these prerequisites.

Model editing and notebook execution require the appropriate permissions. Modeling Copilot, Preview runtime, Code Interpreter, and Microsoft 365 Copilot integration can have additional tenant, licensing, or regional

requirements. Their absence is a feature/access blocker, not evidence of an incorrect analytical answer. Record any section you cannot execute.

Use your own agent names

Choose a short, unique identifier, such as your initials followed by a number. This guide uses AB01 as an example. Replace it consistently with your own identifier; do not use angle brackets in the item names.

Item	Example name
Baseline agent	MfgOps_DA_AB01
AI-ready agent	MfgOps_DA_AIReady_AB01
Multi-source agent	MfgOps_DA_AIReady_AB01_MultiSource

In a shared workspace, model changes affect everyone using those models. Coordinate the model-editing exercises with the facilitator. Do not rename or overwrite somebody else's agent. For a resumed lab, reuse only your own items and inspect their configuration before continuing.

Know the business and the data

The lab uses a fictional industrial manufacturer. It makes about 20 products (pumps, turbines, motors, valves, sensors and spare parts) on 8 production lines in several plants, then sells them to business customers. You do not need a manufacturing background: the terms below cover everything the questions use.

Main tables

Table	One row is...	Used for
ProductionLog	one production run: a line, a shift, a day	planned, produced, good and scrap units; runtime and downtime minutes
Inventory	the stock of a product in a plant on a date (a daily snapshot)	stock on hand and reorder level
Products	a product (SKU)	name, category, list price, cost
Lines / Plants / Assets	a production line / a factory / a piece of equipment	where and on what equipment production runs; Lines[Manufacturer] is the equipment maker
Sales / Customers	an order line / a buying company	revenue and units sold
PurchaseOrders / Vendors	a purchase-order line / a supplier	purchasing spend and supplier delivery
Date	a calendar day	all time filters
Business Measures	no rows; it only holds the measures	the governed KPIs below

Key metrics and terms

Term in the questions	Meaning	Measure
Production quantity	units produced = good units + scrap units	[Production Qty]
Scrap rate (RQX)	share of produced units that were rejected	[Scrap Rate %]
Yield	share of produced units that were good (the opposite of scrap rate)	[Production Yield %]
Day production yield (DPY)	yield excluding the Night shift , not "yield per day"	[Day Yield Pct]
OEE ("reliability" in this lab)	Overall Equipment Effectiveness = Availability × Performance × Quality	[OEE %]
Availability / Performance / Quality	share of time running / produced vs planned / good vs produced	[Availability %], [Performance %], [Quality %]
Downtime	minutes a line was stopped	[Downtime Minutes]
Reorder level	stock level below which a product should be replenished	used by the inventory measures
Inventory risk count	number of daily product-plant snapshots below the reorder level	[Inventory Risk SKU Count]
Total sales / revenue	net sales amount in USD (not list price × quantity)	[Total Sales]
TP	the Pumps and Turbines product categories (in Lab 3: "turbomachinery")	[sls_amt_x] for TP sales
YoY	year over year: the same period one year earlier	—

Two models hold this data:

- ManufacturingOps (**baseline**): deliberately unfriendly, with cryptic names such as custName, prd_yld_day and sls_amt_x, and few descriptions. It shows what an agent does without business context.
- ManufacturingOpsAIReady: the same business with clear names, a description on every measure, and the AI preparation you complete in Lab 1.

In Lab 3 the OpsRefData Lakehouse adds two things the models do not have: the **reasons** for downtime and monthly product sales.

How to read a result

For each question, review the paraphrased request, generated DAX or SQL, query execution result, and final answer. Expand the run-step control beneath the answer; its label and number of steps can vary.

An acceptable numerical answer uses the intended measure, entity, grouping, and period, and agrees with the query output. Fluent prose alone is not sufficient. A query error is not an empty dataset; an empty dataset is not numeric zero.

Use **Clear chat** between independent tests. Keep the same conversation only for exercises explicitly identified as follow-ups.

Each exercise below shows the results observed when this guide was rehearsed end to end, twice, on fresh deployments (3 and 4 October 2026). Where the two runs differed, both are shown: that variability is part of what you are learning to check.

- **Reference result** gives the values from the workshop sample data. Your wording, layout and timing will vary; the measure, period and numbers should match.
- **Known behavior** marks an answer that is deliberately imperfect on the baseline agent. It is the lesson of that exercise, not a broken lab. Run the **Recovery prompt** that follows it to see the correct result.

Establish data-as-of dates before relative questions

Why this matters: "last month" can mean the previous calendar month relative to today, or the previous month relative to the latest loaded fact date. Those are different questions when the imported data is historical. A refresh timestamp tells you when a model was processed; it does not tell you the date of the latest business event.

After connecting each agent to its model in Lab 1, ask:

What are the earliest and latest dates with production records in this model?
Use ProductionLog dates, not the last date in the calendar table. Return one row with both dates. Do not apply a default time window.

1. Expand the run step and verify that the query reads ProductionLog dates, rather than only the calendar table.
2. Write down the minimum and maximum dates for each model separately.
3. Do not assume that the baseline and AI-ready models have identical data.
4. Do not assume that a maximum date proves every earlier day is complete. Inspect monthly counts and consult the facilitator if completeness matters.

Reference result (workshop sample data): the models are imported snapshots, so every deployment returns the same coverage:

Source	First production date	Latest production date
ManufacturingOps (baseline)	1 June 2024	6 July 2026
ManufacturingOpsA IReady	1 June 2024	8 August 2026
OpsRefData Lakehouse (Lab 3)	built relative to the day you run BuildOpsRefData	runs up to that day

Neither model contains data up to today. This is why "last month" in the baseline returns **June 2026**, and why Lab 3 needs explicit dates when it combines the models with the Lakehouse.

If you also ask for "the number of production records", the agent may return the count on each boundary date rather than the all-history total. That is an ambiguity in the question, not a data problem. Use the DAX check below for an exact count.

The check is **optional**. You do not need it for the later questions to work. It tells you whether an unexpected period comes from the agent's interpretation or from data that is not loaded. If the answer is unavailable or ambiguous, open the semantic model's **DAX query view**, create a query, and run this read-only check. If that view is not available to you, ask the facilitator to run it.

```
EVALUATE
ROW(
    "First production date", MIN(ProductionLog[Date]),
    "Latest production date", MAX(ProductionLog[Date]),
    "Production rows", COUNTROWS(ProductionLog)
)
```

For numerical comparisons between agents, choose the same explicit date range within both models' coverage. Use the relative questions as a separate test of time interpretation, rather than silently changing the question to fit the data. A model ending partway through a month should not be presented as a complete month.

Workshop time convention: when no period is supplied, the AI-ready agent will use the latest 30 calendar days **including** the latest production date: from that date minus 29 days through that date. Explicit user dates take precedence. Ambiguous relative phrases should be clarified or accompanied by the exact resolved dates. In production, the business owner must decide whether such phrases are anchored to today or to a data snapshot.

Facilitator cue: "Before trusting the number, we check which dates it describes. The agent cannot report a month that has not been loaded. We want to distinguish data freshness from query correctness."

Lab 1: Creating and optimizing data agents

Goal: build a baseline agent, diagnose ambiguous questions, configure an AI-ready model and agent, and explore runtime and analysis tools.

Step 1: Creating your first data agent

1. In the prepared folder, select **New item > Data agent**. Use the item search if needed.
2. Name the agent MfgOps_DA_AB01, using your own identifier.
3. In **Explorer**, select **Add Data > Data source**.
4. In the OneLake catalog, select the ManufacturingOps **semantic model** in your workspace. Models with the same name from other workspaces can appear in the list: check the **Location** (workspace) column before selecting. Select **Add**.
5. Expand the model and **select all its tables** for this baseline exercise. Selecting the model root shows a warning that the Standard runtime supports up to 25 tables; this model has 13, so select **Continue with Standard**. Confirm that each checkbox is selected; adding a source is not the same as selecting its tables.
6. Run the data-as-of check above. Record this model's production coverage (reference: 1 June 2024 to 6 July 2026).
7. Clear chat and ask the introduction question.

I am new to this agent and the data. Tell me more about it and how to use it.

Expected behavior: the agent describes the connected data and suggests questions. Treat these as suggestions, not a guarantee that every suggested question is in scope or will execute successfully.

What to conclude: The agent describes what it *thinks* it can do from table and column names. That is a starting point, not a list of verified capabilities.

Ask the following one at a time. Clear chat between these independent tests.

Show me the production quantity for the last month.

Expected behavior: a production total with an identifiable period. Inspect whether DAX anchors "last month" to the current date or the latest loaded date.

Reference result: 45,078 units for June 2026 (identical in both rehearsals). The answer may name the month ("Month: 2026-06") or not; the run step paraphrase reads "the most recent fully completed calendar month with production data".

How to read it:

Check	Observation
What you meant	The previous calendar month before today (for example, September 2026 if you run the lab in October).
What the agent did	Used the latest data in the model (data ends 6 July 2026) instead of today, so "last month" became June 2026.
Is the number wrong?	No: 45,078 is the correct June total.
Is the answer right?	No: it answers a different question. Even when it names June, it never says that the model has no data for the month you meant.

Check	Observation
What to conclude	Both conventions ("before today" or "before the latest data") are legitimate, and the model does not say which one applies. The failure is not choosing June; it is not saying so. Relative dates are ambiguous. A trustworthy answer states the exact period used and when the data stops. The AI-ready agent is configured to do this later in the lab.

If different: ask the agent to state the exact dates. Repeat with an explicit month inside the recorded coverage. Do not describe "no records" as zero production, and do not silently relabel an older month as last month.

List the top 5 products by total sales.

Expected behavior: up to five products ranked by a sales measure, with the period and currency stated. The baseline exercise includes sales; the operations-only agent created later will deliberately decline sales questions.

Reference result: HelioGen 3000 Steam Turbine (\$5.91 billion), HelioGen 2000 Gas Turbine (\$4.59 billion), HelioGen 1000 Gas Turbine (\$3.11 billion), AquaFlow 200 Centrifugal Pump (\$856.6 million), AquaFlow 350 Submersible Pump (\$855.7 million), over **all available history**. In one rehearsal the final answer said "across all available data"; in the other, the scope appeared only in the paraphrase and DAX.

What to conclude: The ranking is right, but an answer without its period is incomplete: "top 5" over two years is not "top 5" this quarter. Always check that the period appears in the answer, not only in the query.

If different: inspect the ranking, measure, and period. Product list prices are not total sales. If the query returns fewer than five products, distinguish insufficient data from an omitted result.

Give me a pivot table with product, total sales, quantity, and inventory.

Expected behavior: a table at a declared grain. Check whether "quantity" means units sold or produced, and whether inventory represents one snapshot or an aggregation over multiple snapshots.

Known behavior: a 20-product table with sales, sales quantity and inventory over all available data. Inventory is summed across every daily snapshot (about 62,000–69,000 per product), so it is not a stock level. A text table rather than an interactive pivot is normal.

Recovery prompt (clear chat first):

For June 2026, give me a table with product, total sales and sales quantity. Add inventory on hand from the single latest inventory snapshot in June 2026 only; do not sum inventory across dates. State the snapshot date.

Reference result: June sales with inventory from **one** snapshot (29 June 2026), labelled as such.

What to conclude: Stock is a snapshot: you can add sales across days, but not stock levels. The agent does not know which measures can be summed over time unless the model or the question tells it. A professional-looking table can still be meaningless.

Multi-part question and conversational context

Clear chat, then ask:

Which products have inventory below the reorder quantity? How often does that happen?

Expected behavior: identification of low-stock products plus an explanation of "how often": for example, product-plant-date snapshots below reorder point. Frequency must not be confused with shortfall units.

Known behavior — the unit of "how often" changes between runs. All 20 products were below reorder at least once, but the two rehearsals counted differently:

- Run A used the governed Inventory Risk SKU Count measure: the number of **product-plant-date snapshots** below reorder (for example FlowGuard 10 Control Valve 41, TorqueMax 30 Induction Motor 39).
- Run B computed its own **number of distinct dates** below reorder (for example FlowGuard 10 Control Valve 38 out of 220 dates) and a percentage.

Both are "frequencies", but they are different numbers. Check which unit the answer states.

What to conclude: "How often" needs a defined unit. Even with a governed measure available (Inventory Risk SKU Count), the agent does not always use it, so the same question can return different numbers. Read the stated unit, not just the figures.

Without clearing chat, ask:

For the top three products by frequency, show me the monthly trend as a bar chart.

Expected behavior: the follow-up refers to the same three products and the same definition of frequency. The agent may use one query or several; a fixed number of run steps is not required.

Known behavior: in both rehearsals the **chart showed only the first month** (June 2024) although the query returned many months. In one run the follow-up also **switched the metric** (from snapshot occurrences to distinct days). Conversational context does not guarantee the same definition, and a chart can hide most of its data.

Recovery prompt (same conversation; it names the measure explicitly, so it works whatever unit the first answer used):

Use the governed [Inventory Risk SKU Count] measure: the number of product-plant-date inventory snapshots below reorder quantity. Take the top three products by that measure across all history, breaking ties by product name. Return a table with product, month and that count for every month with data.

Reference result: a product-by-month table for AquaFlow 100 Centrifugal Pump, FlowGuard 10 Control Valve and TorqueMax 30 Induction Motor (FlowGuard 10 = 41, TorqueMax 30 = 39, AquaFlow 100 = 38, first alphabetically among four products tied at 38). The monthly counts add up to each product's total. Prefer the table to the chart.

What to conclude: A follow-up carries over *which* products you discussed, not necessarily *how* you measured them. In multi-turn analysis, restate the metric, the tie-break and the period, and check a chart against its underlying table.

If different: confirm the three product names, date range, and frequency definition explicitly, then retry. If the previous answer did not establish a valid ranking, there is no trustworthy "top three" to carry forward.

Domain-specific KPI question

Clear chat and ask:

For the bottom 20% of products by revenue, do we run into inventory issues? Why?

Expected behavior: inspect how the agent defines bottom 20%, resolves ties, and connects the selected products to inventory measures. It can describe observed low stock or shortfalls.

Known behavior: the DAX comparison is valid, but the answer **invents causes** that no data supports. In the rehearsals these included "lower demand", "conservative stocking policies", "reorder points set too low", "lumpy spare-part demand" and "prioritization bias", once under a heading called "Data-grounded reasoning".

Recovery prompt (clear chat first):

List the products in the bottom 20% by total revenue across all history, with their revenue and their count of inventory snapshots below reorder quantity. Report only what the data shows. If the data cannot explain why, say so instead of suggesting causes.

Reference result: four low-revenue products (Pump Seal Kit, Pump Impeller, SenseLine Temperature Sensor, Turbine Bearing Set) with revenue and inventory risk counts (30, 37, 30, 28), and an explicit statement that the data does not explain the cause.

What to conclude: Asking "why" invites plausible-sounding explanations that the data cannot support. Ask the agent for facts and to say when the data cannot explain a cause; keep evidence and hypotheses separate.

If different: request the revenue period, selected products, and inventory criterion. "Why" does not authorize a causal explanation absent supporting data. Distinguish measured associations from hypotheses.

Facilitator cue: "A plausible answer is the start of the review, not the end. We inspect the measure, time window, and business definition."

Step 2: Improving and optimizing data agent responses

Stay in the baseline agent. Clear chat before each question, expand its run steps, and compare the paraphrased question, generated DAX, and final answer. These are diagnostic exercises: the baseline is not required to fail in a particular way.

Q1: Scrap rate and implicit dates

What is our scrap %?

Expected behavior: look for [Scrap Rate %] and identify the selected time window. The baseline may use all available history, a recent window, or ask for clarification. A correct measure with an unstated period is still ambiguous.

Reference result: 2.35% from the governed [Scrap Rate %] over **all available history**, with that scope stated. This one is correct on the baseline.

What to conclude: When a measure is clearly named (Scrap Rate %), even the baseline uses it correctly. What is still missing is an agreed default period, which the AI-ready model adds.

Next action: retain the measure and make the default period explicit in the AI-ready model's instructions.

Q2: A named KPI without a governed definition

What is our OEE?

Expected behavior: the baseline may infer Overall Equipment Effectiveness, construct an ad hoc calculation, or ask for clarification. On a resumed model, an OEE measure may already exist. Inspect rather than assume.

Known behavior: the baseline model has **no OEE measure**, so the agent writes its own formula inside the query (DEFINE MEASURE . . . [OEE %] = . . .). The formula and the number change between runs: **87.33%** in one rehearsal (availability × schedule attainment × yield, all history) and **92.6%** in another (Performance factor missing, date not stated). Expand the run step to see the improvised definition.

What to conclude: Without a governed definition, the agent invents a formula, and a different one each time. The result looks credible, but it is not *your* OEE. Business KPIs must exist as model measures so the agent reuses them instead of improvising.

Next action: compare any generated formula with the business definition. The AI-ready model contains [OEE %]. Reuse that governed measure instead of assuming that an improvised formula is authoritative.

Q3: Day-shift yield versus daily yield

What is our day production yield for the last six months?

Expected behavior: check whether "day" means **excluding the Night shift** or simply grouping all-shift yield by calendar day. The baseline's [prd_yld_day] implements the former, but its name is not self-explanatory.

Known behavior: the baseline returns **all-shift** Production Yield %, grouped by calendar day or by month (March–July 2026, about 97.6–97.8% per month), not the day-shift measure. In one run it also claimed the model "only has data for these five months", which is a misreading: data starts in June 2024. That is the ambiguity this exercise demonstrates.

What to conclude: The correct measure exists (prd_yld_day), but its name does not say what it means, so the agent cannot find it. Business meaning must be written down in names, descriptions and AI instructions, not only known by the people who built the model.

Next action: the AI-ready model uses [Day Yield Pct] with a description. Put business meaning in model metadata and Prep data for AI, not solely in DAX comments. If the six-month period extends outside coverage, disclose that rather than silently shortening it.

Q4: Internal acronyms

What were the TP sales last week?

Expected behavior: "TP" may be unresolved, misinterpreted, or correctly inferred. In this workshop it means the **Pumps and Turbines** product categories. The cryptic baseline measure [s_ls_amt_x] expresses that sales filter.

Known behavior: the baseline does not know what TP means. Two failure modes were observed:

- It searched product names for "TP", found nothing, got **BLANK** and reported **0 sales**.
- It said no TP designation exists, then reported **total sales for all products** for the last completed week (254,725,180), which answers a different question.

What to conclude: Internal jargon is guessed, and the guess is presented as an answer: either a confident "0" from an empty result, or a substitute total for something you did not ask. Both are worse than "I don't know what TP means".

Next action: inspect the actual category filter and dates. A clarifying question is preferable to a confident answer for the wrong meaning. Later, the AI-ready operations-only agent must decline this **sales** request even though TP itself is a recognized term.

Q5: Business meaning of "machine"

Show me the distribution of scrap rate by machine.

Expected behavior: the agent can interpret "machine" as equipment, production line, or manufacturer. Inspect the grouping column, not just the chart label.

Known behavior: the baseline groups by **asset** (individual equipment) over all history: eight assets between 2.28% and 2.42%, plus three assets (sensors, a transfer pump) with no value. In one rehearsal the chart showed only one category. Check the table in the run step rather than the chart.

What to conclude: "Machine" means different things to different people (equipment, line, manufacturer). The agent picks one; the business must define which. Verify the grouping in the query and table, not the chart.

Next action: for this workshop's AI-ready exercise, "machine" is a business alias for **equipment manufacturer**, mapped to `Lines[Manufacturer]`. This is a workshop-specific convention, not a universal meaning of machine. Use `Lines[Manufacturer]` because production rows reach it through an **active** relationship; the `Assets` relationships to `ProductionLog` and `Lines` are **inactive**, so grouping a production KPI by an `Assets` column does not filter it. The bundled verified answer also uses `Lines[Manufacturer]`.

Facilitator cue: "We are not teaching the agent a universal definition of machine. We are documenting the meaning our business expects."

Step 3: Configuring and optimizing the semantic model and data agent

Define scope

The next agent answers manufacturing operations questions: production, inventory, assets, plants, lines, scrap, yield, and OEE. It does not answer sales, customer, vendor, or purchasing questions. Lab 3 will explicitly add product sales through a different source.

Do not remove sales tables from the underlying shared model to enforce this exercise. Focus the AI schema and agent table selection, and add scope instructions. These controls are not a replacement for permissions or row-/object-level security.

Inspect the AI-ready model

1. Return to the workspace and open the `ManufacturingOpsAIReady` **semantic model**, not its report.
2. Switch to **Editing** mode when authorized.
3. Inspect names and descriptions. Examples include `Customers[Customer Name]`, `Assets[Asset Name]`, `[Day Yield Pct]`, `[Scrap Rate %]`, and `[OEE %]`.
4. Confirm that descriptions explain business meaning and usage, rather than merely repeating the field name.
5. Inspect `[sls_amt_x]` as a metadata example: the description explains TP, although sales will remain outside this agent's scope.

Expected behavior: clearer metadata and governed measures are available. Not every object is necessarily renamed; existing downstream dependencies can make descriptions preferable to renaming.

Check Prep data for AI

1. In the semantic model, select **Prep data for AI**.
2. Open **Simplify the data schema / AI data schema**. The deployment has **already** focused it: the operations tables `Assets`, `Business Measures`, `Date`, `Inventory`, `Lines`, `Plants`, `ProductionLog`, and `Products` are included, while `Customers`, `PurchaseOrders`, `Sales`, `SalesSummary`, `Vendors` and the sales/purchasing measures are excluded. Confirm this rather than changing it. The agent's Explorer still lists all 13 tables: in Step 4 you select the same eight tables in the agent itself.
3. Review **Verified answers**. Look for the scrap-rate-by-machine example. Inspect its underlying grouping, measure, and filters.
4. Confirm that the verified answer groups by `Lines[Manufacturer]`. This matches the rule below. Do not switch it to `Assets[Manufacturer]`: the `Assets` relationships are inactive, so that column does not filter production measures.
5. Open **Add AI instructions**. The bundled text conflicts with this workshop in two places: `RQX = [Quality %]` measure (RQX is scrap rate) and `For all questions related to "machines" use Assets[Manufacturer]` column (that column does not filter production measures). **Select all the**

existing text and replace it with the block below. The block keeps the bundled rules that remain valid, including the CONTAINSSTRING rule for names. It is the instruction set used in both rehearsals that produced the reference results in Step 4 (lightly reformatted for reading).

Use governed measures rather than inventing alternative KPI formulas.

For production questions with no period, use the 30 calendar days ending on the latest ProductionLog[Date], inclusive: latest date minus 29 days through latest date. Do not use the maximum calendar-table date as the data-as-of date.

Explicit user dates take precedence. For ambiguous relative phrases such as "last month", "last week", or "this year", establish the reference date and calendar convention. Return the exact start and end dates used. Disclose incomplete coverage; do not substitute an older period without saying so.

User-specified durations are explicit periods and ALWAYS override the default: six weeks is 42 calendar days, ending on the latest production date unless the user specifies another reference. Use end minus 41 days through end, inclusive, never 30 days for a six-week question.

For every production KPI query, include the actual latest ProductionLog date as a Data As Of result alongside the requested start and end dates. A requested end date later than Data As Of does not mean those later dates contain data.

Day production yield / DPY means yield excluding the Night shift. Use [Day Yield Pct], not all-shift yield grouped by day. Day production yield is a day-shift-only KPI, not a request for a daily time series. When asked to break a KPI down by line for a period, evaluate the governed measure over the entire period and return one row per line. Add daily grouping only when explicitly requested. Do not average daily percentages.

RQX means [Scrap Rate %], not [Quality %].
OEE means [OEE %]. In this workshop, "reliability" is an alias for OEE.
TP refers to Products[Category] values Pumps and Turbines, not Motors.

In this workshop, machine means equipment manufacturer. For a breakdown by machine, group ONLY by Lines[Manufacturer]; do not add asset name, line or location unless the user explicitly asks for that detail. For individual equipment, use Assets[Asset Name]. Use Lines[Manufacturer] through the active Lines-to-ProductionLog relationship: the Assets-to-ProductionLog and Assets-to-Lines relationships are inactive, so grouping a production KPI by Assets attributes does not filter that KPI.

Aggregate at the requested grain before returning results. Do not derive a full-period total or average from a truncated detail result.

Unless the user requests missing groups, exclude rows with BLANK production quantity and BLANK KPI. Preserve actual numeric zeros. Never replace BLANK with zero, including for visualization.

Never calculate sales revenue from Products[Price].
If a query fails, report an execution problem. Do not claim that data is missing unless a successful coverage query supports that statement.

For named-entity columns (e.g., names, places, organizations), use CONTAINSSTRING for partial-text matching by default. Use exact-match filters only when the user explicitly requests a specific entity.

6. Search the saved instructions for Quality % and Assets[Manufacturer]. Neither should appear as a rule (only "not [Quality %]" remains).

7. Save/apply the model changes and close the Prep data for AI pane.

Expected behavior: the AI schema, instructions, and verified-answer metadata express a consistent business interpretation.

Important: semantic-model query-generation rules belong in **Prep data for AI**. Agent-level instructions control orchestration, scope, and presentation; they are not a substitute for model-level DAX guidance.

Verified answers guide DAX generation using their prompts and visual metadata. A data agent does not necessarily return the original Power BI visual. Do not promise an identical chart or guaranteed latency improvement.

Facilitator cue: "We place each rule where it is consumed: business calculations in the model, routing and response behavior in the agent."

Step 4: Testing the optimized semantic model

Create and configure the AI-ready agent

1. Create a new data agent named MfgOps_DA_AIReady_AB01.
2. Add the ManufacturingOpsAIReady semantic model from the same workspace (check the **Location** column in the catalog).
3. The Explorer lists **all 13 tables** of the model. Explicitly select only these eight: Assets, Business Measures, Date, Inventory, Lines, Plants, ProductionLog, and Products. Click **one checkbox at a time** and wait a second for each to save: rapid clicks can be lost when you leave the page.
4. Switch to another Explorer tab and back, expand the source, and confirm the eight checkboxes are still selected. An attached source with no selected tables cannot answer data questions (in an earlier test, this alone dropped the Lab 2 score from 3/3 to 1/3).
5. Open the agent's **Instructions** area and enter the following.

Scope

Answer manufacturing operations questions using the configured AI-ready semantic model: production, inventory, assets, plants, lines, operational performance, trends, and KPIs.

Audience and tone

Help plant managers, operations leaders, production supervisors, inventory planners, analysts, and executives. Be concise, professional, and clear. Explain manufacturing terminology when necessary.

Guidelines

- Decline sales, customer, vendor, purchasing, and purchase-order questions with: "This question is out of scope for this agent. Please ask a manufacturing operations-related question."
- Respect the model's governed KPI definitions and time-period rules.
- When no period is supplied, use the latest 30 days of production data.
- State the requested start/end dates and the actual latest production date separately. If the requested end is later than the data-as-of date, explicitly disclose that records are only available through the data-as-of date. Do not imply full coverage through today.
- For YoY comparisons, always show both current and prior-year start/end dates.
- Clarify ambiguous plant, line, product, or date requests.
- State units, filters, and assumptions. Give the answer first, then supporting evidence.
- Return the requested aggregation grain. For a period broken down by line, request one full-period row per line, not a daily time series. Add daily granularity only when explicitly requested.
- Do not derive complete totals or averages from truncated detail. Requery at the requested aggregate grain.
- Do not invent explanations, missing values, or unsupported causal claims. Distinguish query failure from no matching records and from numeric zero.
- Location means plant location when a location grouping or filter is requested; do not add a location grouping otherwise.

Common terminology

- Machine means equipment manufacturer in this workshop. For by-machine questions request manufacturer-only grouping, not individual assets or lines. Individual equipment means asset name.
- Day production yield / DPY means the governed yield excluding Night shift, evaluated over the requested period. It does not mean daily yield.
- RQX: scrap rate.
- OEE: Overall Equipment Effectiveness; workshop alias "reliability".
- DPMP: defects per million parts.
- YoY / YOY: year over year. MTD: month to date. MOM: month over month.
- TP: Pumps and Turbines product categories.

Period precedence and missing data

Explicit user dates or durations ALWAYS override the default. Apply the latest-30-day default ONLY when no period is specified. Last six weeks means 42 calendar days ending on the latest production date, inclusive; never replace six weeks with 30 days.

Never convert a blank or unavailable KPI into zero, including for a chart.

6. Close/save the instructions pane, reopen it, and confirm the text persisted.

7. Run the production data-as-of check for this model. Record its dates (reference: 1 June 2024 to **8 August 2026**).

8. Clear chat before each independent question below.

Why two layers of instructions? In the first rehearsal, model instructions alone did not fix the day-yield, machine and "this year" questions: the agent's orchestrator rephrased the question (for example into a daily series or an asset grouping) before the model rules applied. Agent instructions control that rephrasing; model instructions control the DAX.

Question 1: Default period

What is our scrap rate?

Expected behavior: [Scrap Rate %] over 30 days including the latest production date, with the resolved dates in the answer.

Reference result: **2.36%**, 10 July to 8 August 2026, with scrap units and production quantity that reconcile.

What to conclude: The documented default (latest 30 days of data) is applied and disclosed. Compare with the baseline: same measure, but now the period is explicit and agreed.

If different: check the model's saved instructions, selected tables, and DAX filter. Retry with explicit start/end dates. Do not report success merely because the response repeats "30 days."

Question 2: Explicit period overrides the default

What is the OEE this year?

Expected behavior: [OEE %], not an ad hoc replacement; the default 30-day period is overridden. The answer should clarify the year/reference date or state it explicitly, and disclose whether it is year-to-date rather than a complete year.

Reference result: **87.18%**, 1 January to 8 August 2026, with "data available through 8 August 2026" stated. Without the data-as-of guidance, the first attempt implied coverage through today.

What to conclude: The governed [OEE %] replaces the improvised formula, and the answer is honest about the data stopping on 8 August. "This year" is answered as year-to-date *with* its real end date.

If different: ask for OEE for an explicit year and end date within coverage. Historical sample data does not become current-year data just because the question says "this year."

Question 3: Day production yield

What is our day production yield for the last six weeks? Break it down by lines.

Expected behavior: [Day Yield Pct], excluding Night shift, grouped by line. The answer identifies the six-week boundaries; "six weeks" can otherwise mean a rolling 42-day interval or six calendar weeks.

Reference result: eight lines, one value each (about 97.4% to 97.7%) for **28 June to 8 August 2026** (42 days). Earlier instruction versions produced 30 days, or a 336-row daily series that hit the 200-row limit; the period-precedence and grain rules fix both.

What to conclude: Getting this right needed rules in both places: the model (which measure, which dates) and the agent (do not turn it into a daily series). Explicit durations must override defaults, and results must be computed at the grain the user asked for.

If different: request 42 days ending on a stated production date. Confirm the measure and line grouping in DAX rather than trusting the answer's title.

Question 4: Out-of-scope request

What were the TP sales last week?

Expected behavior: a refusal, not a sales total. Recognizing TP does not make sales part of this agent's scope.

Reference result: the configured out-of-scope message, without running a query.

What to conclude: Scope instructions control what the agent *tries* to answer. They are not security: users with model access can still query sales elsewhere. Use permissions or row/object-level security for real restrictions.

If different: check that you are in the AI-ready operations-only agent, not the baseline or multi-source agent. Check scope instructions and the focused schema. Never use scope prompts as a substitute for data security.

Question 5: Multiple abbreviations and year-over-year logic

What's the YOY TP reliability?

Expected behavior: TP means Pumps and Turbines; reliability means the governed OEE measure. YoY requires two comparable periods. Asking for the intended period is acceptable.

Reference result: governed OEE for Pumps and Turbines only, with the data-as-of date stated. The **format varied** between rehearsals:

- a single comparison of the latest 30 days with the same window a year earlier: **86.83% vs 86.76% (+0.07 pp)**, both date ranges stated; or
- a **monthly table for the last 12 complete months** (August 2025 – July 2026), each month against the same month a year earlier (for example July 2026: 85.53% vs 87.16%, –1.63 pp).

Both are valid readings of "YoY". In one run, one row's difference was miscalculated (86.07% vs 85.94% shown as "+0.00 pts" instead of +0.13); the next run was correct. Check the arithmetic on a row or two.

What to conclude: Three business terms (YoY, TP, reliability) in one short question are resolved correctly because each is documented. "YoY" itself is still ambiguous (which periods?), so a comparison is only trustworthy when both periods are stated, and the numbers still deserve a quick check.

If different: expand the run steps. A DAX execution error is not evidence that prior-year data is missing. First establish coverage for both periods. Then use this diagnostic follow-up:

First establish the available production dates for Pumps and Turbines.
Ask me to choose comparable current and prior-year periods within coverage.

Then return [OEE %] for each period separately and the difference in percentage points. Do not interpret a failed query as missing data.

If necessary, ask the two period questions separately. Compare results only after both queries succeed; do not calculate a YoY percentage from a missing or zero denominator.

Question 6: Machine/manufacturer interpretation

Show me the distribution of scrap rate by machine.

Expected behavior: manufacturer-level scrap rates consistent with the saved business rule. Inspect whether the query uses `Lines[Manufacturer]`, and whether verified-answer metadata changes that grouping or its filters.

Reference result: six manufacturers for 10 July to 8 August 2026: Fluke 2.60%, GE 2.41%, Mazak 2.38%, NI 2.36%, Marsilli 2.28%, Siemens 2.22%, with matching chart categories. If you see the same rate for every manufacturer, the query grouped by an `Assets` column, which does not filter production.

What to conclude: Instructions must match how the model is actually built. A column that looks right can be disconnected (inactive relationship) and silently return the same value everywhere. Check that a breakdown really varies.

If different: inspect conflicting metadata and ask explicitly:

Show [Scrap Rate %] grouped by `Lines[Manufacturer]` for the latest 30 days of production data. State the dates. I mean equipment manufacturers, not individual assets or production lines.

Different equivalent-looking columns are not automatically interchangeable: check relationships, grain, and totals before accepting the result.

Facilitator cue: "The improvement is clearer, inspectable behavior. We still verify the actual query; instructions are guidance, not a guarantee."

Data agent runtime

The runtime controls the agent's orchestration and query-generation components. Standard and Preview can produce different queries and timings. Preview includes advanced DAX generation; it is not a promise that every query will be faster or error-free.

1. In the AI-ready agent's runtime setting, select **Standard**.
2. Clear chat and ask the following original comparison question.

For each plant, return the smallest ordered set of line-and-shift combinations that reaches at least 80% of that plant's downtime. Include the first combination that crosses 80%. Show plant, line, shift, downtime minutes, rank within plant, share of plant downtime, cumulative share, and cumulative share before the row. Show the result as a table.

3. Record success/failure, total elapsed time, period, returned rows, and DAX.
4. Switch to **Preview**, clear chat, and repeat the identical question.
5. Check correctness before comparing speed. Within each plant, the rows must be ordered by descending downtime, use that plant's full downtime total, and stop at the first row whose cumulative share reaches or exceeds 80%.
6. Inspect tie handling. If downtime values tie, repeat both runtime tests with the same added instruction: Break ties by line name, then shift name, in ascending order. Keep the time range identical.

7. For a fuller comparison, perform ten independent runs per runtime with unchanged configuration and data. Alternate runtimes where practical.

Expected behavior: two sets of measured outcomes, not a prescribed performance percentage. Record unsuccessful runs separately; report both failure rate and the median latency of successful runs. Do not silently drop failures or call a single faster response a benchmark.

Reference result (rehearsal 1: 10 runs per runtime; rehearsal 2: 1 run each; F16 capacity):

Runtime	Queries executed	Fully correct answers	Median time of executed runs
Standard	5 of 10, then 1 of 1 failed ("There's content here I can't work with")	0 of 11	83 s
Preview	10 of 10, then 1 of 1	5 of 10, then 1 of 1	29.5 s; 40 s

Preview was faster and more reliable on this question, but **neither runtime was error-free**. Present it that way: a measured improvement, not a guarantee. Expect Standard to fail on this question.

The period chosen can differ, so check it first. Correct values from a direct DAX check (each plant needs 9 of its 12 line-shift combinations):

Period used	Riverside: 9 rows / plant total	Rheinland: 9 rows / plant total
All history (1 Jun 2024 – 8 Aug 2026)	99,534 / 123,774 min (80.42%)	99,749 / 123,745 min (80.61%)
Latest 30 days (10 Jul – 8 Aug 2026)	3,897 / 4,679 min	3,303 / 4,118 min

Ten runs per runtime take about 30 minutes; in a timed workshop, run one or two each and compare with these tables.

What to conclude: Judge a runtime on correctness first, then speed. Preview was clearly better on this hard question, but neither runtime is guaranteed: complex questions need verification whichever you choose.

If different: if a runtime is unavailable, record that limitation. If a query fails, inspect its error and retry a simpler ranking before returning to the cumulative-threshold question.

Facilitator cue: "We compare correctness and speed together. A faster incorrect answer is not an improvement."

AI-assisted modeling changes

This exercise modifies the **baseline** model. Perform it after the baseline question tests. In a shared workshop, the facilitator should coordinate who applies changes. A resumed, already-edited model will not reproduce the same starting behavior as a clean deployment.

1. Open the ManufacturingOps semantic model, not ManufacturingOpsAIRReady.
2. Switch to **Editing** mode and open **Copilot**.
3. Ask for proposed names without applying changes.

Identify all the name columns in this model. Using the table context and

sample values, propose more descriptive names that clearly reflect the business meaning. Return a table with: Current Name, Proposed Name.

4. If a tool approval appears, review its scope and select **Allow** only for the intended operation.
5. Review the proposed names with the facilitator. Check dependencies in measures, relationships, reports, and verified answers.
6. Once approved, ask:
Approved, make the changes.
7. Reopen the affected tables to confirm the names actually changed. Open the baseline report and check affected visuals. An assistant's completion message is not sufficient evidence that all changes were applied.

Expected behavior: proposed changes followed by persisted, reviewed changes. Copilot can produce partial results or require follow-up.

Reference result: the size of the proposal varies:

- Rehearsal 2: **12** rows, only columns whose name contains "name" (for example custName → Customer Name, Products[Name] → Product Name, ProductionLog[line_name] → Production Line Name), including one no-op (Date[Month Name]). After approval, Copilot applied 11 renames.
- Rehearsal 1: **43** rows, including descriptive fields that are not names. If you get a list like that, narrow it before approving:

Limit the proposal to columns that hold the name of an entity (customer, product, vendor, plant, line, asset and similar). Do not rename IDs, codes, dates, numeric fields or descriptive attributes.

In both rehearsals the approved renames persisted, source mappings and relationships were preserved, and the measures kept working. The baseline report does not use any renamed column, so its visuals are unaffected. Reopen the tables to confirm.

What to conclude: Copilot speeds up modeling work, but its proposal is not deterministic: one run proposed 43 renames, another 12. A human scopes the change, approves it and checks it was saved.

Optional: Add descriptions

Ask:

This semantic model will be used by a Fabric data agent to answer manufacturing operations questions. Add descriptions based on the following guidelines.

DO: Add a description to every visible table, column, and measure. Keep descriptions concise and front-load the key meaning within the first 200 characters. Front-load preferred usage, disambiguation, and units. Make implicit knowledge explicit. Do not restate the field name or add DAX logic. Sample values if necessary to learn the domain and context. Include expected grain where useful. For calculation groups, describe the items and their use.

DON'T: Generate descriptions purely from AI without business context. AI-only descriptions tend to restate what the model structure already shows. Always validate descriptions with the user or a domain expert. Do not contradict descriptions across related fields.

If Copilot returns proposals for review, review them before saying:

Update the descriptions.

Expected behavior: useful, persisted descriptions, reviewed against the data and business meaning. The 200-character target is a concise-writing convention here, not a claim that every product surface has the same hard retrieval limit.

Reference result: Copilot works for several minutes and behaves in one of two ways:

- **Proposes, then waits** (rehearsal 1): it flagged **27** objects as uncertain ("for review"). Apply only the confident ones (109):

Apply only the descriptions you are confident about. Skip every item you flagged for review.

- **Applies directly** (rehearsal 2): no proposal and no approval step. The model went from 24 to 118 described objects. Its summary claimed it had also described the five cryptic demo measures (sls_amt_x, gm2_pct, po_ok_flagish, prd_yld_day, inv_rsk_u), but **none of those five was saved**, and the meaning it gave for po_ok_flagish did not match its DAX.

Either way, open a few objects, including the five demo measures in the **Ambiguous Names Demo** folder, and check what was actually saved. If descriptions were applied without review, you can undo them through the model's version history.

What to conclude: AI-written descriptions are a draft, and the assistant's summary of what it changed can be wrong. Check what was saved, apply only what is correct, and leave uncertain items for a domain expert.

Finish by reviewing business-friendly names, descriptions, synonyms, sensible hierarchies, model relationships, security, the AI schema, verified answers, and AI instructions. Do not introduce new relationships or security changes solely to complete this checklist.

Code Interpreter

1. Return to your **AI-ready agent**.
2. Select **Add tools > Code interpreter** in the toolbar, then select **Add to data agent** in the confirmation dialog. Open the **Tools** tab and check that Code interpreter is listed: without the confirmation, the tool is not enabled.

3. Clear chat and ask:

Show me a pivot table of products vs. the last six months by reliability. Put products on rows and months in columns.

4. Check the OEE measure, six-month boundaries, and coverage. Keep this conversation open and ask:

Show me a heatmap.

5. Expand the analysis steps. Inspect the Python code, input data, and output.

Expected behavior: a heatmap based on the preceding product/month result, not a new and unrelated query. Missing cells should remain distinguishable from zero. Inspect how percentages and color scales are represented.

Reference result:

- Pivot question: governed OEE for 20 products × **March to August 2026**, with data available through 8 August. **August is partial** (1–8 August): one rehearsal labelled it "Aug 2026 MTD", the other just "August". The result can be a grid or a list.
- Heatmap: a products-by-months **image** (with download links in rehearsal 2), and a Python step in the run steps.

- **Without the tool enabled**, the agent still answers "Show me a heatmap" with an emoji-coloured text table. It looks like a heatmap, but no Python ran.

What to conclude: Code Interpreter adds analysis and charts the semantic model cannot produce, on the same governed numbers. Check that Python really ran (an image and a code step, not coloured text), and check partial periods: a month labelled "August" may contain only 8 days.

If different: if no Python tool ran, do not present the result as a Code Interpreter demonstration. Confirm the tool is enabled and ask: Use Code Interpreter to render the preceding table as a heatmap. Preserve missing cells and label the OEE percentage scale.

Optional: Detrending and FFT

Clear chat and ask:

Detrend the daily scrap rate for Pump Impeller for the last two months, run FFT on it, and identify any dominant modes.

Expected behavior: data retrieval followed by Python analysis. Check the product, daily measure, time range, observation count, missing dates, detrending method, and FFT frequency units.

Reference result: two steps (query, then Python), with the data-as-of date stated. The **method differed** between rehearsals:

- Rehearsal 1: the two complete months June–July 2026, **interpolated** the days without production, removed a linear trend and found a dominant cycle of about **15 days**.
- Rehearsal 2: 9 June – 8 August 2026 (61 calendar days, only 31 with data), **no interpolation**, modes expressed in production days (strongest about **7.75 production days**), with an explicit warning that a calendar-day cycle cannot be inferred.

What to conclude: Python makes advanced statistics easy to request, but every method rests on assumptions, and the agent picks them for you. Same question, different gap handling, different "dominant cycle". Read how missing days were treated before trusting any pattern, and treat it as a lead to investigate, not a root cause.

If different: an empty or irregular series may not support the requested analysis. Request coverage and sampling checks before accepting any result. Do not silently fill gaps with zero. A spectral peak is a candidate pattern, not proof of a manufacturing root cause.

Facilitator cue: "Python extends what we can calculate, but it does not remove the need to check the data and the statistical assumptions."

Lab 2: Programmatic evaluation of data agents

Goal: calibrate an LLM judge against human labels, evaluate your operations-only AI-ready agent, and inspect the results in MLflow.

Run both notebooks **inside Fabric**. The workshop's Responses API evaluation workflow is not an instruction to run the notebooks locally. Allow extra time for environment startup, package installation, and service calls.

Step 1: LLM-as-Judge calibration

1. Open the public [judge calibration workbook](#) for review.
2. Inspect `calibration_development` and `calibration_holdout`. Development examples are for rubric refinement; holdout examples are for an independent acceptance decision. The lab provides labels, so you do not need to invent them.
3. Open `JudgeCalibration` in your Fabric workspace.
4. Inspect its **Configuration** cell. Keep the supplied model, rubric, data reference, and acceptance thresholds unless the facilitator instructs otherwise. Note the judge model and `JUDGE_REGISTRY_EXPERIMENT`.
5. Select **Run all**, and wait for execution to finish. If package installation requests a session restart, follow the notebook instruction before continuing.
6. Review the development/holdout results and the notebook's registration decision. Do not register a failed judge by lowering thresholds to make the exercise pass.
7. Open the MLflow run linked by the notebook. Inspect the rubric, model, human-label agreement metrics, data reference, and registration tags.

Expected behavior: the notebook creates or reuses the evaluation Lakehouse, loads the labeled workbook, evaluates the candidate judge, logs evidence, and registers a champion **only if its acceptance rules are met**.

Reference result: champion judge registered (model `gpt-5.1`, 34 development and 18 holdout examples) in about 2 minutes. Agreement with the human labels was **97.1%** (development) and **94.4%** (holdout, Cohen's kappa **0.886**) in one rehearsal, and **100%** on every metric in the other. If you run the notebook as a scheduled job, its status can show **Completed** or **Cancelled** (the last cell stops the session on purpose); check the MLflow run and its `judge_status = champion` tag instead.

What to conclude: Before an AI grades another AI, check it against human judgement. 94–100% agreement on unseen examples justifies using this judge for automated tests; a judge below the threshold would not be registered.

If different: distinguish authentication, model availability, capacity, package, or storage errors from poor agreement with human labels. Stop before agent evaluation if no approved judge is available.

Facilitator cue: "Before asking an AI to grade another AI, we check that its grading agrees with the human standard."

Step 2: Evaluate your AI-ready agent

1. Open `EvaluateDataAgent`.
2. Find **Select the Data Agent**. Set the following to the **exact display name of your own agent**, not the illustrative `..._SAP` default.

```
DATA_AGENT_NAME = "MfgOps_DA_AIReady_AB01"
```

3. In **Configuration**, confirm `GT_MODEL = "ManufacturingOpsAIRReady"`. Leave `REFRESH = "No"` unless you intentionally need a refresh. Refreshing during a comparison can change the answer key.
4. Confirm that the agent is the operations-only agent from Lab 1. The workshop evaluation includes a sales refusal check; it is not the same policy as the multi-source agent in Lab 3.
5. Review the evaluation workbook path used by your notebook. The source version accompanying this guide uses `eval_set_L400.xlsx` and selects three cases when `TEST_MODE = True`. Inspect `TEST_CASE_IDS`; do not assume the workbook itself contains only three rows.
6. Confirm that the eight source tables remain selected and the model and agent instructions are saved.
7. Select **Run all**. The workflow generates numerical ground truth from the semantic model and evaluates policy questions against expected behavior.
8. Review the overall score and **Per-question review** under the results section. Open the MLflow run.

Expected behavior: inspectable per-question answers, DAX/run steps, ground truth or policy expectations, judge reasoning, and metrics. Completion of the notebook is not equivalent to 100% answer accuracy.

Reference result: overall, factual and behavioral accuracy **1.0 (3/3)**, 0 infrastructure errors, with the judge loaded from the registry. In an earlier run where the agent's tables were not selected, the score was 1/3: only the refusal passed.

What to conclude: Automated evaluation turns "it seems to work" into a repeatable score. It also catches configuration mistakes: one unselected setting dropped the score from 3/3 to 1/3 while the agent still looked fine in chat.

If different: when a refusal succeeds but data questions fail, first inspect source selection, data access, query errors, and the period used. A policy refusal can succeed without successful data retrieval.

Correct one issue at a time, label the next run using the notebook's `STAGE` field, and rerun. Preserve the first run for comparison. The supplied source uses the agent's sandbox stage for evaluation: do not assume this proves the published endpoint is identical.

Checkpoint: do not move on with an unexplained failed factual answer. Record the failed case and its cause, or ask the facilitator for help. A three-case evaluation is a learning exercise, not production certification.

Lab 3: Adding multiple data sources

Goal: add Lakehouse data for downtime reasons and product sales, inspect the resulting routing and queries, then publish and consume the agent.

This explicitly broadens scope: **product sales become allowed through OpsRefData**. Customer-level data, vendors, and purchasing remain outside scope.

Step 1: Build the Lakehouse source

1. Open BuildOpsRefData.
2. Use the supported Python 3.11 or 3.12 notebook runtime for the supplied notebook. Do not convert it to an unrelated Spark runtime solely for this exercise.
3. In a shared workshop, ask the facilitator whether OpsRefData is already prepared. The notebook rebuilds shared data and SQL objects; participants should not all rebuild it concurrently.
4. If it needs building, select **Run all** and wait for the final result.
5. Return to the workspace and open the OpsRefData Lakehouse.
6. Switch to its **SQL analytics endpoint**.
7. Expand the fda schema and confirm the Downtime_Reasons and Sales_Orders views, plus the Downtime_By_Line_Type table-valued function.
8. Open **New SQL query** in that endpoint and run:

```
SELECT 'Downtime_Reasons' AS [Object], COUNT(*) AS [Rows]
FROM [fda].[Downtime_Reasons]
UNION ALL
SELECT 'Sales_Orders', COUNT(*)
FROM [fda].[Sales_Orders];
```

```
SELECT *
FROM [fda].[Downtime_By_Line_Type]('Assembly')
ORDER BY [Downtime Minutes] DESC;
```

9. Record source coverage:

```
SELECT MIN([Date]) AS [First Date], MAX([Date]) AS [Latest Date]
FROM [fda].[Downtime_Reasons];
```

```
SELECT MIN([Order Date]) AS [First Sales Month],
       MAX([Order Date]) AS [Latest Sales Month]
FROM [fda].[Sales_Orders];
```

Expected behavior: both views contain queryable data, and the function returns the Assembly downtime-reason breakdown. The function is all-history and has no date parameter; use the view for date-filtered analysis.

Reference result: Downtime_Reasons 75,490 rows and Sales_Orders 1,160 rows. The Assembly function returns Equipment Failure 72,110, Changeover 31,949, Planned Maintenance 20,752, Material Shortage 19,160 and Operator Error 15,608 minutes.

Important — dates differ from the models: BuildOpsRefData generates Lakehouse data **up to recent days**, not up to the models' dates. In the rehearsals, downtime reasons ran from 1 June 2024 to 3 October 2026 and sales months to 1 October 2026. The semantic models stop on 6 July / 8 August 2026. So "latest 30 days" can mean a different period in each source. Keep this in mind for Step 3.

What to conclude: Different sources rarely have the same data freshness. Before combining them, know each one's dates.

If different: SQL endpoint metadata can lag table creation. Inspect the notebook's first failed cell, wait for endpoint metadata refresh, and rerun the affected step under the facilitator's guidance. Do not treat missing SQL objects as successfully configured sources.

Important grain distinction: Sales_Orders contains monthly product/plant facts; Order Date is the first day of the sales month. It is not a daily transaction feed. A latest month key does not prove that the month is complete.

Step 2: Create the multi-source agent with the SDK

1. Open CreateMultiSourceDataAgent in the **same workspace** as your AI-ready agent and OpsRefData.

2. Inspect **Step 1 - Parameters** and update:

```
BASE_DATA_AGENT_NAME = "MfgOps_DA_AIRReady_AB01"
MULTI_SOURCE_AGENT_NAME = f"{BASE_DATA_AGENT_NAME}_MultiSource"
SEMANTIC_MODEL_NAME = "ManufacturingOpsAIRReady"
LAKEHOUSE_NAME = "OpsRefData"
PUBLISH_CHANGES = True
```

3. Before running, review the notebook's publication behavior and the target name. PUBLISH_CHANGES = True publishes in Step 9. If you are not ready to publish, set it to False and stop before the published MCP test in Step 10.

4. Run cells from **Step 0** through **Step 8** in order. Allow package installation and source discovery to finish.

5. Read the output: the base agent must exist; the eight model tables and three Lakehouse objects must be selected.

6. Open the newly created ..._MultiSource agent in Fabric and inspect its sources, agent instructions, and Lakehouse **Setup** guidance/examples.

Expected behavior: your base agent remains available, and the separate multi-source agent has both sources configured.

Important: inspect your notebook version. The supplied source creates or reuses the derived agent and applies its own source selection, instructions, and examples. It is not a guarantee of a complete clone of every custom base setting, runtime, or tool. Recheck any customizations you need.

The notebook broadens sales scope and replaces agent instructions. Do not keep the operations-only "all sales are out of scope" rule on this multi-source agent. Conversely, do not broaden the Lab 1 agent.

Inspect and clarify time handling

The supplied notebook contains snapshot-relative time defaults, including "this year" based on the selected source's latest date. Inspect those rules: they can differ from the calendar-relative meaning a user expects.

For the cross-source exercises, use **explicit matching periods**. When comparing daily production with monthly sales, use calendar months whose coverage is confirmed in both sources. Do not claim that simply taking the earlier maximum date proves complete coverage, or prorate a monthly sales row into daily sales.

Never join daily downtime detail directly to monthly sales detail. Aggregate each side independently to the same plant/month grain first. Keep missing groups visible when the question requires them, and do not infer causality from a correlation.

Step 3: Publish and test the MCP endpoint

1. Return to **Step 9 - Review and publish**. Review the staging source names and saved settings.
2. If publication is intended and authorized, run Step 9 with `PUBLISH_CHANGES = True`. Otherwise, publish your agent manually after reviewing it and only then proceed.
3. Run **Step 10 - Query the published agent through MCP**. Keep the notebook session open until you finish the later post-publication checks.
4. Inspect each test result, not just `is_error`. A successful MCP transport response can still contain a refusal, incomplete answer, or query failure.

The supplied tests cover the following:

Test	Question / intended route
Semantic model	"What was the scrap rate in the latest 30 days?" - governed model measure.
Lakehouse	"Which product had the highest sales?" - actual revenue from Sales_Orders.
Both sources	"Which line had the most downtime minutes, and what were its leading downtime reasons, using the same latest 30-day period?" - align periods and distinguish totals from reasons.
Function	"Across all available history, what is the downtime-reason breakdown for Assembly lines?" - the all-history function is appropriate.

Expected behavior: a published endpoint can be initialized and queried using the signed-in identity, and each answer meets its analytical criteria. Keep authentication tokens out of screenshots, notebook exports, and public repositories.

Reference result: the cell completes and its `is_error` assertion passes. Reading the answers:

Test	What you will see
Semantic model	2.36% , 10 July to 8 August 2026 — same as the Lab 1 agent. Correct.
Lakehouse	The top product by revenue, but the period varies : HelioGen 3000 Steam Turbine, 165,376,000 "across all available order dates" (rehearsal 2, twice), or HelioGen 2000 over the Lakehouse's own latest 30 days (rehearsal 1). Both are correct for the period used; check which period the answer states.
Both sources	Check it. Line A1 - Pump Assembly, 1,588 minutes, 10 July – 8 August 2026, reasons adding up to 1,588: this aligned answer came back in 3 of 3 runs in rehearsal 2. In rehearsal 1, the answer mixed periods : the total came from the model (ending 8 August) but the reasons from the Lakehouse's September–October data (adding up to 1,833), while claiming a single period. In both cases <code>is_error</code> was False.
Function	All-history Assembly breakdown, identical to the SQL function above. Correct.

This is the key Lab 3 lesson: **a transport success is not an answer check**. For the combined question, check that the reasons add up to the line's total. To remove the risk, rerun it with explicit dates, in the agent's Test chat or by replacing that entry in `MCP_TEST_QUESTIONS`:

For 2026-07-10 through 2026-08-08 inclusive, which line had the most downtime

minutes, and what were its leading downtime reasons in that same period?
Apply exactly these dates to both sources and state them.

Reference result: Line A1 - Pump Assembly, **1,588** downtime minutes; reasons Equipment Failure 713, Changeover 322, Planned Maintenance 207, Material Shortage 190, Operator Error 156 (sum 1,588).

What to conclude: "No error" is not "correct". With relative periods, the agent *can* combine numbers from two different time windows and present them as one; it does not do so every time, which makes it harder to spot. With explicit dates, the two sources reconcile exactly (reasons add up to 1,588). For cross-source questions, fix the period yourself.

If different: check whether the agent was published, whether your identity has source access, and whether the endpoint has become available. A saved draft does not update the published endpoint. Do not change tenant permissions just to hide a failed test.

Step 4: Creator Assistant - Build agent with AI

Use this exercise on the **multi-source agent** and its Lakehouse source. The assistant can propose configuration changes, but you must verify each requested change was persisted.

1. Select **Build agent with AI**.

2. Enter the original multi-change request:

Can you update the agent instructions and the OpsRefData data source instructions, and add a few-shot example so that when a user asks about the turbomachinery category, the query filters for Pumps and Turbines, but not Motors? Please sample the values first to confirm.

3. Review the sampled category values (Motors, Pumps, Sensors, Spare Parts, Turbines, Valves).

4. When satisfied, say:

Apply these changes.

5. The assistant writes **one artifact per turn** and asks for confirmation each time. Whenever it shows a draft and says *Reply "save"*, reply:

save

6. Inspect **three separate locations** before continuing:

Location	Required persisted content
Agent instructions	Turbomachinery means Pumps and Turbines, excluding Motors; sales routes to OpsRefData.
OpsRefData source instructions	SQL filters [Category] IN ('Pumps', 'Turbines') for turbomachinery.
OpsRefData example queries	A saved question/query pair demonstrates that category filter.

Expected behavior: all three requested changes exist in their correct locations. A confirmation message is not evidence of all three writes.

Known behavior: a single "Apply" does **not** write all three changes. In rehearsal 2 the conversation took four turns:

You send	The assistant
Apply these changes.	Shows draft agent instructions and asks you to reply "save". Nothing is saved yet.
save	Saves the agent instructions, then drafts the OpsRefData source instructions on its own.
save	Saves the source instructions and offers the example query.
Now add the turbomachinery few-shot example query to the OpsRefData source.	Drafts the example (it also rewrites the existing examples).
save	Saves the examples.

In rehearsal 1, each change needed its own request. If one is missing, ask for it explicitly:

Now update the OpsRefData data source instructions with the turbomachinery rule.

Now add the turbomachinery few-shot example query to the OpsRefData source.

In both rehearsals, all three locations contained the change at the end.

What to conclude: The assistant's "done" message is not proof. Check every place a change should land.

If different: ask for the missing change individually, or use the source's **Setup** editor to add it. Preserve existing routing, grain, and date rules. For a simple category-only example, add the following question and SQL pair.

Across all available sales history, what are revenue and units sold for turbomachinery, excluding Motors?

```
SELECT SUM([Revenue]) AS [Revenue],
       SUM([Units Sold]) AS [Units Sold]
FROM [fda].[Sales_Orders]
WHERE [Category] IN ('Pumps', 'Turbines');
```

This example explicitly requests all available history. Do not reuse its unfiltered dates for questions asking for a particular month or year.

7. Ask the assistant:

Test this change and verify that it works as expected.

It runs its own test query. In rehearsal 2 it tested the **latest 30 days** (Turbines 16,818,000 / 87 units, Pumps 1,336,600 / 180 units) plus a check that Motors exist in the data, so its numbers do not match the all-history totals below. That is expected: compare like with like.

8. Return to the normal chat (select **Test data agent** in the toolbar; it may be under **More (...)** on a narrow window). Clear chat and ask the same all-history question.

9. Run the SQL above directly in the OpsRefData SQL analytics endpoint. Compare revenue and units, source, categories, and period.

10. Inspect a category breakdown if totals disagree:

```
SELECT [Category], SUM([Revenue]) AS [Revenue],
       SUM([Units Sold]) AS [Units Sold]
FROM [fda].[Sales_Orders]
GROUP BY [Category]
```

ORDER BY [Category];

Expected behavior: the turbomachinery result equals the Pumps plus Turbines totals for the same period and excludes Motors. Do not copy a numerical total from a different environment as the answer key.

Reference result: revenue **419,704,600** and **7,118** units, filter [Category] IN ('Pumps', 'Turbines'), identical in the direct SQL query, the Test chat, the published agent, the MCP endpoint and Microsoft 365 Copilot, in both rehearsals. The category breakdown shows Motors at 33,728,200 / 9,026, correctly excluded. Your totals can differ if your Lakehouse was built on another date; what must match is the agent's answer and your direct SQL query.

What to conclude: The business term "turbomachinery" now consistently means Pumps + Turbines, and the agent's answer equals a direct database query: that is the test of a correct configuration.

Facilitator cue: "We asked for three changes, so we inspect three saved settings. Then we compare the agent's answer with a direct source query."

Step 5: Description and final publication

1. Ask Creator Assistant:

Generate a concise description of what this data agent does. Focus on the specific business domain and the questions it answers. Avoid generic, reusable descriptions. Make it specific to this data agent's unique purpose and capabilities. Do not use bullet points and do not mention table names, column names, datasets, schemas, or other technical implementation details. Use clear language that accurately describes its value.

2. Review and copy the description. It should describe both manufacturing operations and product sales without claiming unsupported functionality. The assistant also offers to "save" it; you can ignore that and paste it into the Publish dialog instead.

3. Select **Publish**. The description field may be **prefilled with a summary of your recent changes** rather than a capabilities description: replace it with the description from step 1.

4. If you will use Microsoft 365 Copilot, append the original output-handling guidance:

The output from the data agent should be delivered as-is, without summarizing, rephrasing, or adding extra interpretation or insight.

5. Publish the latest changes. Keep **Also publish to Microsoft 365 Copilot** off unless you are completing the next step and have the required authorization.

6. Return to the still-open notebook session and rerun the MCP query cell with the same turbomachinery question used in Step 4, replacing or adding an entry in MCP_TEST_QUESTIONS. Do not rerun the configuration-writing cells: they can overwrite the Creator Assistant changes.

7. Compare the published answer with the direct SQL and draft answer, using identical scope and dates. When finished, run the notebook's final session stop cell.

For step 6, replace only the MCP_TEST_QUESTIONS list in notebook Step 10 with the following, then rerun that Step 10 cell:

```
MCP_TEST_QUESTIONS = [
    (
        "turbomachinery",
        "Across all available sales history, what are revenue and units sold "
        "for turbomachinery, excluding Motors?",
    ),
]
```

Expected behavior: the published agent reflects the latest configuration. The description guidance can reduce Microsoft 365 Copilot rephrasing; it does not guarantee verbatim output.

Reference result: the published Test view and the MCP endpoint both returned 419,704,600 / 7,118 with Motors excluded, `is_error = False`.

What to conclude: Publishing creates the version others use. Testing the published version, not just the draft, confirms that what you validated is what users get.

Step 6: Consume the agent in Microsoft 365 Copilot

Complete this step when the workshop account has the required Microsoft 365 entitlement and Copilot extensibility enabled. Use the **same account and tenant** as Fabric. Publishing to the Agent Store is an additional action, not a prerequisite for the preceding Fabric and MCP exercises.

1. In your agent's **Publish** dialog, review the description and enable **Also publish to Microsoft 365 Copilot**.
2. Publish and open Microsoft 365 Copilot with the same account.
3. Open **Agents / Agent Store**, locate your agent by its exact name, and open it. Navigation labels can vary.
4. If it has not appeared, refresh the agent list or expand navigation and allow propagation time. If still unavailable, ask the facilitator to check entitlement and extensibility settings.
5. Start a new conversation directly with the agent, or select it using @.
6. Ask the **same** turbomachinery question used for SQL, draft, and MCP.
7. Compare revenue, units, categories, and period. Do not require identical prose; Microsoft 365 Copilot has its own orchestration layer.

Expected behavior: the agent is discoverable for the authorized user and returns a source-grounded answer consistent with the equivalent direct query.

Reference result: in both rehearsals the agent appeared in the Agent Store search within 10–20 minutes of publishing; search for part of its name. The search result shows the full name with a generic description ("Declarative agent that uses Data Agent to answer questions"); the agent's details card can cut the name to 30 characters (MfgOps_DA_AIRReady_AB01_MultiSo). Select **Open**. The answer took 40–80 seconds: 419,704,600 revenue and 7,118 units, Pumps and Turbines only.

What to conclude: The same governed answer reaches users in Microsoft 365 Copilot. The wording can differ because Copilot adds its own layer; compare the numbers and filters, not the prose.

If different: confirm agent identity, account, tenant, publication state, underlying source access, and time range. A missing Agent Store entry is not a failed DAX or SQL calculation.

Do not share the agent externally as part of this exercise. Agent Store publication does not grant recipients access to underlying sources. Microsoft 365 consumption can process results under Microsoft 365's data handling terms and outside Fabric's compliance boundary; follow your organization's policy before enabling it.

Workshop completion checklist

Area	Evidence to retain in your own approved environment
Baseline agent	Selected source tables, data-as-of dates, and inspected responses.
AI-ready agent	Eight selected tables, consistent model/agent guidance, governed KPI queries, and correct sales refusal.
Relative questions	Declared reference dates and exact resolved periods; no silent substitution of missing months.
Runtime	Recorded correctness and timing; unsuccessful runs remain visible.
Modeling	Approved names/descriptions persisted, with dependent content checked.
Code Interpreter	Python execution and chart inputs inspected, or feature blocker recorded.
Evaluation	Approved judge plus per-question agent results and MLflow evidence.
Multi-source	Both sources selected, routing inspected, source grain and periods understood.
Creator Assistant	All three requested configuration changes persisted and directly checked.
Publication	Matching question/scope checked in draft and published MCP.
Microsoft 365 Copilot	Matching factual result, or a clearly recorded access/feature blocker.

You have completed the learning path when you can explain both successful answers and unresolved failures. Do not mark an unexecuted optional feature as passed.

Troubleshooting

Symptom	First action
Setup notebook fails with HTTP 403 at the DataFolder parameter step	Fix that cell's Authorization header to "Bearer " + power_bi_token, then rerun (see Before you start).
Agent loses table selections	Click one checkbox at a time, wait for each to save, then reopen the Explorer to confirm.
Every manufacturer shows the same scrap rate	The query grouped by an Assets column (inactive relationship). Use Lines[Manufacturer].
Six-week question uses 30 days, or returns a daily series	Check the period-precedence and grain rules in both model and agent instructions.
Combined downtime question mixes periods	Expected with relative periods: the Lakehouse runs to today, the models do not. Ask with explicit dates.
Stuck in Build agent with AI mode	Select Test data agent in the toolbar (or under More (...)).
"You can only open 20 items at a time" when opening an item	Close some open items in the left navigation bar (each opened item stays there), then retry.
"Show me a heatmap" returns coloured text, not an image	Code Interpreter is not enabled: add it again and confirm Add to data agent .
Creator Assistant shows a draft but nothing changes	Reply save: it writes one artifact per turn and waits for confirmation.
Agent not found in Microsoft 365 Copilot	Wait 10–20 minutes after publishing and search part of the name.
"Last month" returns an unexpected month	Inspect the DAX reference date and the fact table's coverage; ask with explicit dates.
Baseline and AI-ready totals differ	Align measure, filters, date range, and underlying data coverage before comparing quality.
AI-ready agent cannot answer data questions	Confirm that tables are selected, the source is accessible, and the query executed.
RQX gives quality rather than scrap	Remove the conflicting model instruction; confirm the query uses [Scrap Rate %].
Day yield includes Night shift	Inspect [Day Yield Pct] selection and model-level guidance.
YoY answer says "no data" after a failed query	Treat it as a query failure; establish coverage with a successful independent query.
Notebook cannot find an agent	Replace the sample name with the exact agent display name in that workspace.
Notebook evaluation succeeds but score is low	Read each failed case and its ground truth; notebook execution is not answer correctness.

Symptom	First action
Creator Assistant says done, but behavior is unchanged	Inspect agent instructions, source instructions, and few-shot examples separately.
Sales behavior changes after Lab 3	Expected only on the multi-source agent, which routes product sales to OpsRefData.
SQL objects are absent after building the Lakehouse	Inspect the failed build/metadata step; wait for endpoint synchronization before retrying.
Published result differs from draft	Republish the reviewed configuration and use the same question, source, and period.
Copilot wording differs from Fabric	Compare facts and filters; the extra orchestrator can rephrase results.

Sources and license

Based on the MIT-licensed [Microsoft Fabric Data Agent Workshop](#). Lab and notebook reference: [commit 136bcf196a7f935fb030bd9fe81ad0e133d2fbf3](#). Check your deployed notebook cells if their version differs.

Product guidance:

- [Semantic model best practices for data agents](#)
- [Data agent configurations](#)
- [Data agent query-generation best practices](#)
- [Consume a data agent from Microsoft 365 Copilot](#)

This is a community workshop guide, not a product warranty or a guarantee of deterministic AI responses. Microsoft product names remain their owners' trademarks. This guide does not imply Microsoft sponsorship.

MIT license

Copyright (c) Microsoft Corporation.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.